

Student Management System

Python + CSV Based Academic Record Management

Project Report

**Made By Rudransh
26BAI11002**

1. Introduction

A menu-driven Python application for maintaining student academic records. It uses CSV file storage and the tabulate library to provide a simple console-based record management system.

Core operations include adding, displaying, searching, and deleting student records, with automatic calculation of total marks, percentage, and grade.

2. Problem Statement

Manual student record management can be time-consuming and error-prone. Calculating results repeatedly and locating individual records is inconvenient.

The proposed system provides a lightweight computerized solution for storing student details, calculating results, searching records, displaying data, and deleting records.

3. Functional Requirements

Add Student; Display Students; Search Student; Delete Student; Automatic Total/Percentage/Grade Calculation; Input Validation; CSV Persistence.

4. Non-functional Requirements

Usability, reliability, maintainability, portability, and persistence are the main non-functional goals.

5. System Architecture

User/Console → Python Application → students.csv. The Python layer handles menu control, validation, calculations, search, display and deletion. CSV provides persistent storage.

6. Design Diagrams

Use Case: User interacts with Add, Display, Search, Delete and Exit operations. Workflow: Start

→ Menu → Select Operation → Perform Operation → Show Result.

Sequence: User selects an operation; Python validates/processes the request; CSV is read or written; the application returns a result.

Class/Component: create_file(), add_student(), find_grade(), display_students(), search_student(), delete_student().

ER: A single Student entity represents the CSV record, with Roll No as the primary identifier and fields for name, five subjects, total, percentage and grade.

7. Design Decisions & Rationale

CSV was selected for simple persistent storage without database configuration. Functions divide responsibilities. The console interface keeps the project easy to demonstrate. Percentage is total divided by five because there are five subjects, each out of 100.

8. Implementation Details

Technologies: Python 3, csv module and tabulate. The file students.csv contains one header row followed by student records. Input marks are restricted to 0–100, duplicate roll numbers are rejected, and grade thresholds are implemented in find_grade().

9. Screenshots / Results

The application displays records in a formatted grid. Sample records are shown below.

Roll No	Name	English	Maths	Physics	Chemistry	Computer	Total	Percentage	Grade
1	Rudra	85	92	78	88	95	438	87.60	A
2	Jai	90	95	97	99	98	479	95.80	A+
3	Aarav	78	85	82	80	88	413	82.60	A
4	Vihaan	92	88	90	94	91	455	91.00	A+
5	Aditya	75	72	80	78	85	390	78.00	B
6	Arjun	68	74	70	76	72	360	72.00	B
7	Kabir	88	81	85	79	90	423	84.60	A
8	Rohan	65	70	68	72	75	350	70.00	B
9	Ishaan	55	62	58	65	60	300	60.00	C
10	Dev	42	48	45	50	47	232	46.40	E

10. Testing Approach

Testing covers valid input, marks outside the allowed range, non-numeric input, duplicate roll numbers, searching existing/non-existing students, and deletion of records.

11. Challenges Faced

Input validation, duplicate detection, CSV consistency after deletion, readable multi-column output, and correct result calculations.

12. Learnings & Key Takeaways

Practical Python functions, CSV file handling, exception handling, conditional logic, CRUD-style console application design, and basic system architecture.

13. Future Enhancements

Update/edit feature, sorting, printable mark sheets, SQLite/MySQL storage, authentication, graphical/web UI, attendance and analytics.

14. References

Python Standard Library / csv documentation; tabulate documentation; project source code.

Grade Scale

Percentage	Grade
90–100	A+
80–89	A
70–79	B
60–69	C
50–59	D
40–49	E
Below 40	F